

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1

Total Marks: 100

Annexure A

Assignment

Code of Conduct: I __S.Poojitha(192320076)_____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: __S.Poojitha_____

Course Code & Name	CSA11 – Object Oriented Analysis and Design
Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Disaster Resilience and Emergency Response Platform (AIDERP)
Course Outcome(s) – CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented

	Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 3 – Good Health and Well-being SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities SDG 13 – Climate Action

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an AI-Enabled Disaster Resilience and Emergency Response Platform (AIDERP). The platform is intended to support communities, emergency coordinators, hospitals, police/fire services, volunteers, utility teams, and local administrators during floods, cyclones, fires, earthquakes, heatwaves, and other emergencies.

The system shall collect and integrate information from citizen reports, IoT sensors, weather feeds, emergency service updates, geospatial data, and infrastructure status. An intelligent risk-analysis module shall assess the severity and likely impact of an incident and recommend suitable response actions. The platform shall provide role-based interfaces for citizens, emergency operators, field responders, medical teams, volunteers, and administrators.

Citizens shall be able to report incidents, share their location, request emergency assistance, receive verified alerts, identify nearby shelters, and obtain evacuation guidance. Emergency operators shall validate incidents, prioritize cases, coordinate response teams, monitor resource availability, and issue alerts. Field responders shall receive assignments, update incident status, share observations, and request additional

resources. Hospitals and shelters shall update capacity and availability.

The system may use IoT-enabled flood, fire, air-quality, temperature, and structural sensors. A geospatial intelligence module shall support incident mapping, safe-route identification, shelter selection, and resource allocation. AI-based prioritization may consider incident severity, population exposure, accessibility, available resources, and real-time sensor information.

Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop appropriate UML models including Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall evaluate object responsibilities and apply suitable design principles and patterns to achieve modularity, maintainability, reusability, scalability, resilience, and secure information exchange.

Constraints

The system shall clearly define the system boundary, actors, objects, classes, attributes, operations, relationships, assumptions, and external interfaces. UML diagrams shall be consistent with one another, and the operations identified in the Class Diagram shall support the interactions represented in the Sequence and Activity Diagrams.

The design shall demonstrate appropriate use of encapsulation, abstraction, inheritance, polymorphism, cohesion, coupling, responsibility assignment, and interface-based design. Suitable design patterns shall be selected and justified based on the identified design problems.

The system shall distinguish between verified and unverified incident information and should address availability, privacy, authentication, authorization, fault tolerance, and graceful degradation during network or infrastructure failures. AI recommendations shall be treated as decision support rather than unquestionable commands.

The final model shall document assumptions and constraints and ensure that all UML models are complete, consistent, unambiguous, traceable, feasible, scalable, and maintainable.

Tools Can Be Used

- StarUML
- Visual Paradigm
- Draw.io
- PlantUML

- Papyrus Tools
- ArgoUML
- Enterprise Architect
- Lucidchart

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good / Satisfactory / Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the disaster-response domain and accurately identifies actors, objects, system responsibilities, functional and non-functional requirements.	Good: Most actors, objects and requirements identified with minor gaps. Satisfactory: Basic analysis provided but incomplete. Needs Improvement: Major domain elements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	10	Correctly identifies actors, use cases, system boundary and relationships; diagram is complete and logically justified.	Good: Mostly correct with minor modelling errors. Satisfactory: Basic actors/use cases identified but relationships need improvement. Needs Improvement: Incomplete or inconsistent modelling.
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	15	Classes, attributes, methods, associations, aggregation/composition, inheritance and dependencies are accurately identified and justified.	Good: Mostly correct with minor errors. Satisfactory: Basic classes and relationships identified with limited detail.

			Needs Improvement: Major structural elements are missing or inappropriate.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity and State Transition Diagrams accurately represent emergency workflows and are consistent with use cases and classes.	Good: Mostly correct with minor inconsistencies. Satisfactory: Basic behavioural diagrams provided but some interactions/states are incomplete. Needs Improvement: Diagrams are missing, unclear or inconsistent.
UML Architectural Modelling & Design Evaluation (CO3)	15	Package, Component and Deployment Diagrams clearly represent modules, dependencies, services, edge/IoT devices, cloud infrastructure and external systems.	Good: Architecture is mostly correct with minor omissions. Satisfactory: Basic architecture shown but lacks clarity. Needs Improvement: Architectural models are incomplete or inappropriate.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are well assigned; cohesion, coupling, encapsulation, abstraction and suitable patterns are clearly justified and applied.	Good: Principles and patterns appropriately applied with minor limitations. Satisfactory: Some principles identified but application is limited. Needs Improvement: Principles, responsibilities or

			patterns are absent/incorrect.
Consistency, Traceability & Overall Design Quality (CO3)	10	All UML models are mutually consistent; requirements trace to use cases/classes; design is complete, resilient, secure, scalable and maintainable.	Good: Largely consistent and traceable with minor gaps. Satisfactory: Some traceability demonstrated but links are missing. Needs Improvement: Major inconsistencies exist.
Reflection, SDG Relevance & Learning Outcomes	10	Provides insightful reflection on design decisions, challenges, OOAD concepts learned and strong links to the selected SDGs.	Good: Good reflection and SDG connection. Satisfactory: Reflection is present but generic. Needs Improvement: Reflection is missing, superficial or unrelated.

Total: 100

Deliverables

1. Problem Analysis and Requirement Summary
2. Functional and Non-Functional Requirements
3. Actor Identification
4. Use Case Descriptions
5. Use Case Diagram
6. Object and Class Identification
7. Class Diagram
8. Sequence/Interaction Diagram(s) for key emergency scenarios
9. Activity Diagram

10. State Transition Diagram
11. Package Diagram
12. Component Diagram
13. Deployment Diagram
14. Object Responsibility Analysis
15. Design Axioms/Principles and Design Pattern Justification
16. Assumptions, Constraints, Security and Privacy Considerations
17. Requirement-to-Use Case/Class Traceability Matrix
18. Implementation/Prototype and Results
19. Reflection on Design Decisions, SDG Relevance and Learning Outcomes
20. GitHub Upload

Suggested Key Scenarios for UML Modelling

- Citizen reports a flood/fire emergency and requests assistance.
- AI risk engine prioritizes incidents and recommends response resources.
- Emergency operator verifies an incident and dispatches a response team.
- Responder accepts an assignment and updates the incident status from En Route to Resolved.
- Citizen receives an evacuation alert and obtains a recommended safe route to a shelter.
- Hospital or shelter updates available capacity during a large-scale emergency.
- IoT sensor detects abnormal conditions and automatically creates a preliminary incident.

ANSWERS

1. Problem Analysis and Requirement Summary

The AI-Enabled Disaster Resilience and Emergency Response Platform (AIDERP) is designed to support communities, emergency coordinators, hospitals, police and fire services, volunteers, utility teams, and local administrators during floods, cyclones, fires, earthquakes, heatwaves, and other emergencies. The platform integrates citizen reports, IoT sensors, weather feeds, emergency service updates, geospatial data, and infrastructure status. An AI-based risk-analysis module evaluates incident severity and likely impact and recommends suitable response actions. The system provides role-based interfaces so that each stakeholder can perform appropriate emergency activities.

The main engineering problem is the lack of a unified platform for collecting emergency information, assessing risk, coordinating response resources, and communicating verified information. AIDERP addresses this by providing incident reporting, verification, AI-assisted prioritization, dispatch, resource monitoring, safe-route identification, shelter information, capacity updates, and emergency alerts.

2. Functional and Non-Functional Requirements

ID	Functional Requirement
FR1	User registration, authentication and role-based access.
FR2	Citizen incident reporting with description, type and location.
FR3	Collection of IoT sensor and external feed information.
FR4	Automatic creation of preliminary incidents from abnormal sensor readings.
FR5	Incident verification and classification as verified or unverified.
FR6	AI-based risk and severity assessment.
FR7	Incident prioritization and resource recommendation.
FR8	Response-team and resource assignment.
FR9	Responder assignment acceptance and status updates.

FR10	Verified emergency alerts and assistance notifications.
FR11	Nearby shelter identification and safe evacuation route guidance.
FR12	Hospital and shelter capacity updates.
FR13	Incident, resource, alert and audit record management.
FR14	Administrative user, role, configuration and monitoring functions.

2.1 Non-Functional Requirements

Category	Requirement
Availability	Emergency functions should remain available during partial failures.
Security	Authentication, authorization, encryption and audit logging are required.
Privacy	Personal and location information must be protected and accessed only when necessary.
Performance	Critical incident and alert processing should have low response time.
Scalability	The system should support increasing users, incidents, sensors and regions.
Reliability	Emergency transactions and status changes should be consistent and recoverable.
Usability	Interfaces should be simple, accessible and suitable for stressful situations.
Maintainability	The design should use modular components with high cohesion and low coupling.
Resilience	Retries, queues, caching, backups and graceful degradation should be provided.

3. Actor Identification

Actor	Responsibilities
Citizen	Report incidents, request assistance, share location, receive alerts, find shelters and obtain evacuation guidance.
Emergency Operator	Validate incidents, prioritize cases, coordinate teams, monitor resources and issue alerts.
Field Responder	Receive assignments, accept tasks, update incident status, share observations and request resources.
Medical Team / Hospital	Coordinate medical response and update hospital capacity.
Volunteer	Accept suitable response tasks and provide field observations.
Shelter Manager	Update shelter capacity and availability.
Administrator	Manage users, roles, configuration, audit information and system monitoring.
IoT Sensor	Transmit environmental and structural readings.
Weather / Emergency Feed	Provide external weather and emergency information.
GIS Service	Provide mapping, location, route and geospatial information.
Notification Service	Deliver emergency alerts and notifications.

4. Use Case Descriptions

Use Case	Actor	Description
Report Incident	Citizen	Submits incident type, description, location and optional evidence.
Request Assistance	Citizen	Requests emergency support and provides current location.

Receive Verified Alert	Citizen	Receives verified emergency information from the system.
Verify Incident	Emergency Operator	Reviews incident information and verifies or rejects it.
Analyze Risk	System / AI Engine	Calculates severity, exposure, accessibility and recommended resources.
Prioritize Incident	Emergency Operator / AI	Ranks incidents according to risk and available information.
Dispatch Team	Emergency Operator	Assigns an available response team and required resources.
Update Incident	Field Responder	Updates progress and incident status.
Find Safe Route	Citizen	Obtains a recommended route to an appropriate shelter.
Update Capacity	Hospital / Shelter Manager	Updates beds, shelter spaces and availability.
Monitor Sensors	System / Operator	Receives and evaluates sensor readings.

5. Use Case Diagram – Answer

System Boundary: AIDERP.

Citizen → Report Incident, Request Assistance, Receive Verified Alert, Find Shelter, Get Safe Route.

Emergency Operator → Verify Incident, Analyze Risk, Prioritize Incident, Dispatch Team, Manage Resources, Issue Alert.

Field Responder → View Assignment, Accept Assignment, Update Incident, Request Resources.

Hospital/Shelter Manager → Update Capacity.

Administrator → Manage Users, Manage Roles, Monitor System.

IoT Sensor → Send Sensor Data → Monitor Sensors → Create Preliminary Incident.

Weather Feed, GIS Service and Notification Service are external systems connected to the relevant AIDERP services.

6. Object and Class Identification

The major objects/classes are User, Citizen, EmergencyOperator, FieldResponder, Volunteer, Administrator, Incident, Location, Sensor, SensorReading, RiskAssessment, ResponseTeam, Assignment, Resource, Shelter, Hospital, Alert, Route, WeatherReport and AuditLog. Supporting service classes include AuthenticationService, IncidentService, RiskAnalysisService, DispatchService, ResourceService, GISService and NotificationService.

7. Class Diagram – Answer

Class	Attributes	Methods
User	userId, name, email, phone, role, status	login(), logout(), updateProfile()
Citizen	citizenId, emergencyContact	reportIncident(), requestAssistance(), receiveAlert()
Incident	incidentId, type, description, location, severity, status, verified	create(), verify(), updateStatus(), assignTeam()
Sensor	sensorId, type, location, status	sendReading(), checkHealth()
SensorReading	readingId, value, unit, timestamp	validate(), detectAbnormality()
RiskAssessment	assessmentId, severityScore, exposure, confidence, recommendation	calculateRisk(), recommendResources()
ResponseTeam	teamId, type, members, availability	acceptAssignment(), updateStatus()
Assignment	assignmentId, assignedAt, status, priority	accept(), update(), complete()
Resource	resourceId, type, quantity, status, location	allocate(), release(), updateAvailability()

Shelter	shelterId, name, location, capacity, availableSpaces	updateCapacity(), checkAvailability()
Hospital	hospitalId, name, location, capacity, availableBeds	updateCapacity(), requestMedicalSupport()
Alert	alertId, message, severity, timestamp, verified	create(), publish(), cancel()
Route	routeId, origin, destination, safetyScore	calculate(), recommend()
Location	latitude, longitude, address	distanceTo(), validate()
AuditLog	logId, actorId, action, timestamp	record(), search()

Relationships: User is generalized into Citizen, EmergencyOperator, FieldResponder, Volunteer and Administrator. Incident is associated with Location and may contain multiple SensorReadings, RiskAssessments, Assignments, Alerts and AuditLogs. ResponseTeam is associated with Assignment. Assignment connects Incident and Resource. Shelter and Hospital maintain capacity information. Route uses Location and GIS services.

8. Sequence Diagram – Citizen Reports a Flood and Requests Assistance

1. Citizen → AIDERP Interface: Submit incident and location.
2. Interface → IncidentService: createIncident().
3. IncidentService → IncidentRepository: saveIncident().
4. IncidentService → RiskAnalysisService: analyzeRisk(incident).
5. RiskAnalysisService → GISService: assessExposureAndAccessibility().
6. RiskAnalysisService → IncidentService: return risk score and recommendation.
7. IncidentService → NotificationService: notify operator.
8. Emergency Operator → IncidentService: verifyIncident().
9. IncidentService → DispatchService: requestDispatch().
10. DispatchService → ResourceService: findAvailableResources().
11. DispatchService → ResponseTeam: createAssignment().
12. NotificationService → Citizen/Responder: send status and assignment notifications.

9. Activity Diagram – Answer

Start → Citizen opens emergency reporting → Enter incident details → Capture/validate location → Submit report → System records incident → AI risk analysis → Check

information sufficiency → If insufficient, request additional information → If sufficient, operator verifies incident → Decision: verified? → If unverified, monitor/store incident → If verified, prioritize incident → Identify resources → Dispatch team → Responder accepts assignment → En Route → On Scene → Resolve incident → Notify citizen → End.

10. State Transition Diagram – Answer

NEW → SUBMITTED → UNDER_ANALYSIS → VERIFIED or UNVERIFIED.

VERIFIED → PRIORITIZED → DISPATCHED → EN_ROUTE → ON_SCENE → RESOLVING → RESOLVED → CLOSED.

If new evidence shows continuing danger, a resolved or closed incident may be reopened according to operator authorization.

11. Package Diagram – Answer

Package	Contents
User & Access	Users, authentication, authorization and roles.
Incident Management	Incident reports, verification and lifecycle.
Risk Intelligence	AI analysis, risk scoring and recommendations.
Response Coordination	Teams, assignments, dispatch and resources.
Geospatial Services	Location, mapping, routes and shelter selection.
IoT & External Integration	Sensors, weather feeds and emergency feeds.
Alert & Communication	Verified alerts and notifications.
Health & Shelter	Hospital and shelter capacity.
Administration	Users, configuration, audit and monitoring.
Data Access	Repositories, database, cache and message queues.

12. Component Diagram – Answer

Major components are Citizen Web/Mobile App, Operator Console, Responder App, Admin Console, API Gateway, Authentication Service, Incident Service, Risk Analysis Service, Dispatch Service, Resource Service, GIS Service, Notification Service, Sensor Gateway, Data Integration Service, Database, Cache, Message Queue and Audit Service. External components include weather providers, GIS providers and notification providers.

The API Gateway provides secure access to application services. The Sensor Gateway receives IoT information. The Risk Analysis Service processes incident and sensor data. Dispatch and Resource services coordinate response. GIS supports safe-route and shelter selection. Notification Service sends verified alerts.

13. Deployment Diagram – Answer

Client nodes: citizen smartphone/web browser, responder mobile device, operator workstation and administrator workstation. Network layer: Internet/mobile network and secure API gateway. Application layer: load balancer, authentication service, incident service, AI risk engine, dispatch service, GIS service, notification service, sensor gateway and message broker. Data layer: relational database, cache, audit storage and backup storage. External nodes: weather service, GIS service, SMS/push notification service and IoT gateways.

HTTPS/TLS should be used for communication. Application services can be replicated horizontally. Monitoring, health checks, database backup and failover mechanisms improve availability and resilience.

14. Object Responsibility Analysis

Object/Class	Responsibility
Incident	Maintain incident details, verification and lifecycle state.
RiskAssessment	Maintain risk score, confidence, factors and recommendations.
DispatchService	Select suitable teams and assign resources.
ResponseTeam	Manage team availability and assignment status.
Resource	Track quantity, location and allocation state.
GISService	Calculate routes, distances, exposure and shelter suitability.

NotificationService	Deliver verified alerts and operational notifications.
SensorGateway	Validate and ingest sensor messages.
AuditLog	Record sensitive system actions for traceability.
AuthenticationService	Authenticate users and manage secure sessions/tokens.

15. Design Axioms, Principles and Design Pattern Justification

Encapsulation is achieved by keeping object data private and exposing controlled operations. Abstraction is used through interfaces such as NotificationProvider and MapProvider. Inheritance represents common User behavior and specialized roles. Polymorphism allows multiple notification or GIS providers to implement a common interface.

High cohesion is maintained by giving each service a focused responsibility. Low coupling is achieved by communicating through interfaces and events rather than direct implementation dependencies.

Observer Pattern: useful for notifying operators, responders and citizens when incident states change. Strategy Pattern: useful for selecting different risk-analysis or route-scoring algorithms. Factory Pattern: useful for creating incident or notification objects. Adapter Pattern: useful for integrating different external weather, GIS and messaging APIs. Repository Pattern: separates domain logic from database implementation. Retry and Circuit Breaker patterns improve resilience when external services fail.

16. Assumptions, Constraints, Security and Privacy Considerations

Assumptions: authorized users provide valid information; sensors and external providers are available when possible; GIS information is available for supported regions; and emergency operators remain responsible for critical decisions.

Constraints include network outages, inaccurate sensor readings, incomplete reports, limited resources, external API failures and rapidly changing disaster conditions.

Security measures include strong authentication, role-based authorization, TLS, secure password hashing, token expiry, input validation, rate limiting, audit logging, least-privilege access, secure API credentials and regular backups.

Privacy measures include data minimization, restricted access to location information, retention policies, access logging and appropriate deletion or anonymization. Incident

information must distinguish verified and unverified information. AI recommendations are decision support and should not be treated as unquestionable commands.

17. Requirement-to-Use Case/Class Traceability Matrix

Requirement	Use Case	Main Classes/Components
FR1	Authenticate User	User, AuthenticationService
FR2	Report Incident	Citizen, Incident, IncidentService
FR3	Monitor Sensors	Sensor, SensorReading, SensorGateway
FR4	Create Preliminary Incident	SensorGateway, IncidentService
FR5	Verify Incident	EmergencyOperator, Incident
FR6	Analyze Risk	RiskAssessment, RiskAnalysisService
FR7	Prioritize Incident	Incident, RiskAnalysisService
FR8	Dispatch Team	Assignment, ResponseTeam, DispatchService
FR9	Update Incident	FieldResponder, Assignment, Incident
FR10	Receive Alert	Alert, NotificationService
FR11	Find Safe Route	Route, GISService, Shelter
FR12	Update Capacity	Shelter, Hospital
FR13	Maintain Records	Repository, AuditLog, Database
FR14	Manage Users	Administrator, User, AuthenticationService

18. Implementation / Prototype and Results

A prototype can be developed as a responsive web-based system with separate interfaces for citizens, operators, responders and administrators. A REST API backend can manage domain operations, while a relational database stores persistent information. The prototype can integrate an AI risk-analysis module, GIS services, IoT ingestion and notification services.

Important screens include Login/Register, Citizen Dashboard, Report Incident, Assistance Request, Alerts and Shelter Map, Operator Dashboard, Incident Verification, Dispatch and Resource Board, Responder Assignment and Status Update, Hospital/Shelter Capacity, and Administrator Management.

Expected results are successful incident creation, risk assessment, verification, resource assignment, responder status tracking, alert generation, shelter selection and capacity updates. The modular design allows additional sensors, disaster types, regions and AI models to be added later.

19. Reflection on Design Decisions, SDG Relevance and Learning Outcomes

The system was divided into focused modules so that incident management, AI analysis, dispatch, GIS, communication and external integration can be developed and maintained independently. Role-based interfaces were selected because different stakeholders require different information and permissions.

The modelling process improves understanding of object identification, class responsibilities, relationships, UML behavioral modelling, architectural modelling and design patterns. Sequence and activity diagrams connect requirements with system behavior, while the state diagram clarifies the incident lifecycle.

SDG 3 – Good Health and Well-being is supported through emergency assistance, medical coordination and shelter information. SDG 9 – Industry, Innovation and Infrastructure is supported through AI, IoT and resilient digital infrastructure. SDG 11 – Sustainable Cities and Communities is supported through coordinated emergency response and evacuation planning. SDG 13 – Climate Action is relevant to floods, cyclones, heatwaves and other climate-related emergencies.